

| 2026-02-18

| Снижение размерности

Справедливо, что качество моделей часто **растет с увеличением числа признаков**

НО! Сложность и ресурсоемкость моделей растет еще быстрее

Tip

Хорошим примером такого масштабирования являются современные языковые модели: модель на 8 миллиардов параметров уступит модели на 70 миллиардов, а та уступит модели на 621 миллиард (и т.д.).

Но уже сам порядок чисел — миллиарды параметров — говорит о том, что для работы, обучения и модификации подобных моделей нужны большие объемы вычислительных мощностей, оперативной памяти (и/или видеопамати GPU) и данных для тренировки и валидации.

Во многом можно поставить знак равенства между большой моделью и достаточно хорошей моделью — см., например, эссе [The Bitter Lesson](#), посвященное этой проблеме.

| Проклятие размерности (Curse of Dimensionality)

1. **Эффект убывающей отдачи (diminishing returns)**: в какой-то момент при росте числа признаков точность почти не растет, а стоимость обучения и инференса растет резко.
2. **Различная значимость признаков**: не все признаки одинаково полезны для модели.

Под **«проклятием размерности»** понимают совокупность эффектов при росте размерности признакового пространства:

1. **Данные становятся разреженными (sparse)**: все больше «пустых» измерений.
2. **Расстояния между точками выравниваются**: метрики расстояния теряют информативность.
3. **Растет риск переобучения**: модель подгоняется под шум в данных, оптимизация становится сложнее.

Следовательно, разумно не просто добавлять признаки, а уметь **отбирать** и **сжимать** их до компактного представления без потери важной информации.

| Методы снижения размерности

Снижение размерности — преобразование исходного набора признаков размерности N в набор меньшей размерности K , $K < N$, который сохраняет «существенную» информацию для решения задачи (кластеризация, классификация, регрессия).

Цели снижения размерности

1. Ускорить обучение и инференс моделей.
2. Снизить риск переобучения.
3. Упростить визуализацию (2D/3D-проекции многомерных данных).
4. Уменьшить шум и коррелированные признаки.
5. Подготовить компактные признаки для последующих методов (*классификаторов, кластеризации*).

Подходы к снижению размерности

1. **Отбор признаков (feature selection)**: выбор подмножества исходных признаков. *Методы фильтрации, wrapper-методы, embedded-методы.*
2. **Извлечение признаков (feature extraction)**: построение новых признаков как функций от исходных. Линейные методы: *PCA, LDA*. Нелинейные: *t-SNE, UMAP, автоэнкодеры* и др.

Метод главных компонент (Principal Component Analysis, PCA)

Линейный метод снижения размерности, который создает новую систему координат так, чтобы максимальная дисперсия данных приходилась на первые оси — **главные компоненты**.

Суть можно описать так:

1. Имеется облако точек в многомерном пространстве признаков размерности N
2. Мы ищем направление (вектор), вдоль которого *разброс точек (дисперсия) максимален* — это будет **первая главная компонента**.
3. **Вторая главная компонента** — направление, **ортогональное первой**, с *максимальной оставшейся дисперсией*.
4. И так далее до N компонент (для N исходных признаков)

Если взять только первые K компонент, при этом $K < N$, то мы получим «сжатое» представление данных, сохраняющее большую часть вариации.

Применение PCA в различных задачах

1. В задачах **регрессии** позволяет убрать мультиколлинеарность (когда независимые переменные сильно коррелируют друг с другом) и улучшить устойчивость моделей.
2. В задачах **классификации** PCA помогает визуализировать классы и подавить шум.

3. В **кластеризации** PCA часто используют как шаг предобработки для уменьшения размерности перед k -средних.

Алгоритм PCA

Рассмотрим матрицу данных $X \in \mathbb{R}^{m \times n}$, где m — число объектов, а n — число признаков.

1. **Центрирование данных:** вычитаем среднее по каждому признаку.
2. **Матрица ковариации:** $C = \frac{1}{m-1} X^T X$
3. **Собственное разложение (eigendecomposition):** Находим собственные значения (eigenvalues) λ_i и собственные векторы (eigenvectors) v_i ковариационной матрицы C . [Собственный вектор — Википедия](#)
4. **Сортировка компонент:** сортируем пары (λ_i, v_i) по убыванию λ_i .
5. **Выбор K главных компонент:** берем первые K собственных векторов в матрицу W_K .
6. **Преобразование данных:** $Z = XW_K$ — новая матрица данных размерностью $m \times k$

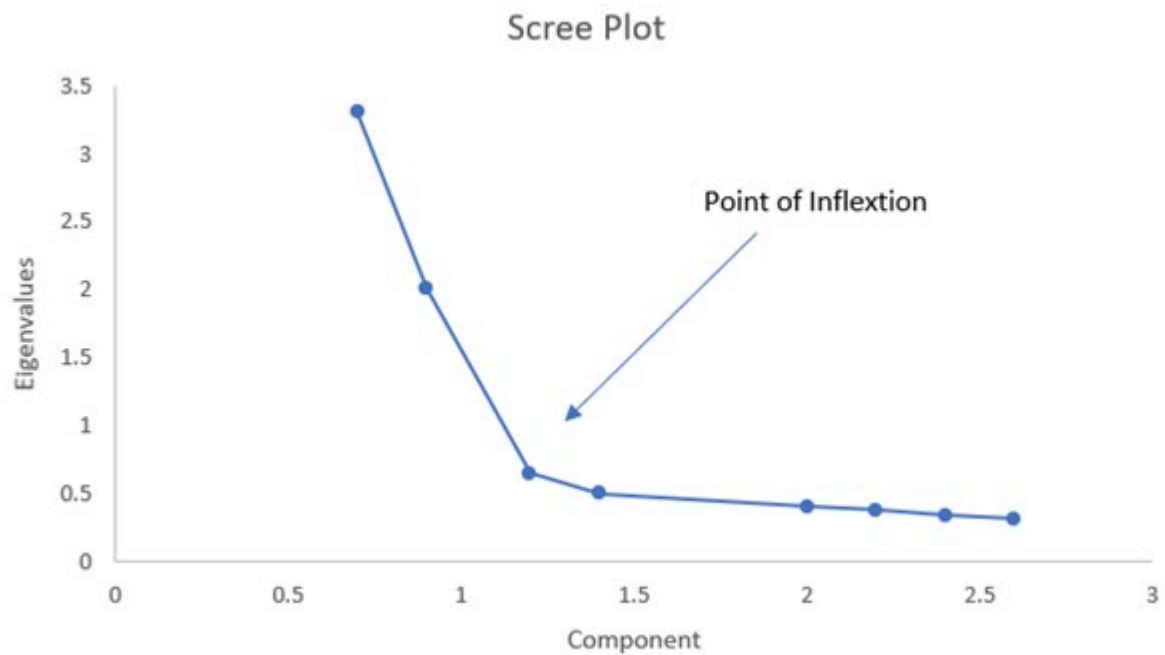
Матрица ковариаций

$$S = \begin{pmatrix} \sigma_{x1x1} & \sigma_{x1x2} & \cdots & \sigma_{x1xq} & \sigma_{x1y1} & \cdots & \sigma_{x1yp} \\ \sigma_{x2x1} & \sigma_{x2x2} & \cdots & \sigma_{x2xq} & \sigma_{x2y1} & \cdots & \sigma_{x2yp} \\ \sigma_{x3x1} & \sigma_{x3x2} & \cdots & \sigma_{x3xq} & \sigma_{x3y1} & \cdots & \sigma_{x3yp} \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \sigma_{ypx1} & \sigma_{ypx2} & \cdots & \sigma_{ypxq} & \sigma_{ypy1} & \cdots & \sigma_{ypy p} \end{pmatrix}$$

$$S = \left(\begin{array}{c|c} S_{11} & S_{12} \\ \hline S_{21} & S_{22} \end{array} \right)$$

Определение числа компонент

1. **Метод локтя:** график собственных значений или суммарной дисперсии и поиск «перелома».



2. **Порог по доле объясненной дисперсии** (например, 90–95%).**

Доля объясненной дисперсии для первых K компонент:

$$\text{ExplainedVariance}(K) = \frac{\sum_{i=1}^K \lambda_i}{\sum_{j=1}^n \lambda_j}$$

Tip

См. пример в Экселе <https://real-statistics.com/multivariate-statistics/factor-analysis/principal-component-analysis/>

Матрица ковариации:

	A	B	C	D	E	F	G	H
1	Y	X ₁	X ₂			Y	X ₁	X ₂
2	15	26	2,7		Y	59,33333		
3	17	33	2,9		X ₁	82,6	146	
4	19	39	3,6		X ₂	4,526667	6,993333	0,408
5	22	48	4		ковариационная матрица			
6	35	55	4,1					
7	8	25	2,4					
8	23	40	3,5			Y	X ₁	X ₂
9	11	31	3		Y	1		
10	6	22	2,2		X ₁	0,887471	1	
11	19	45	3,5		X ₂	0,920024	0,906103	1
12	17	41	2,9		корреляционная матрица			
13	9	23	2,3					
14	16	39	3					
15	23	60	3,6					
16	30	58	4,3					
17								

Исходные данные:

	A	B	C	D	E	F	G	H	I	J
1	Teacher Evaluation									
2										
3		Expect	Entertain	Comm	Expert	Motivate	Caring	Charisma	Passion	Friendly
4	1	2	8	1	4	7	5	4	4	8
5	2	4	8	5	3	7	7	7	6	6
6	3	2	8	2	3	6	7	1	3	7
7	4	4	8	4	2	8	7	7	5	7
8	5	3	8	5	4	8	8	7	6	7
9	6	4	7	3	3	6	6	1	4	7
10	7	4	8	4	2	6	4	5	4	7
11	8	4	8	3	3	7	5	4	4	7
12	9	2	8	2	2	7	6	1	4	7
13	10	4	8	3	4	8	7	4	4	8

Статистические величины

	A	B	C	D	E	F	G	H	I	J
125		Expect	Entertain	Comm	Expert	Motivate	Caring	Charisma	Passion	Friendly
126	mean	3.708333	8.166667	3.45	3.383333	6.708333	6.008333	4.841667	4.725	7.3
127	stdev	1.266211	0.570026	1.649293	1.014047	0.956146	1.368897	2.589558	0.95233	0.588403
128	skew	0.138578	0.558794	0.609682	0.250119	0.030993	-0.23512	0.047501	-0.07387	0.322172
129	kurt	-0.07509	1.224573	-0.37097	0.790481	-0.09199	-0.42618	-0.92189	-0.24984	0.134885

Матрица ковариаций на нормализованных данных

	AH	AI	AJ	AK	AL	AM	AN	AO	AP	AQ
3		Expect	Entertain	Comm	Expert	Motivate	Caring	Charisma	Passion	Friendly
4	Expect	1.603291	-0.12745	0.165966	0.154762	-0.17822	-0.18242	-0.07178	-0.15651	-0.2563
5	Entertain	-0.12745	0.32493	0.420168	0.028011	0.133053	0.091036	0.665266	0.205882	0.134454
6	Comm	0.165966	0.420168	2.720168	0.254622	0.451681	0.239916	3.197899	0.578571	0.006723
7	Expert	0.154762	0.028011	0.254622	1.028291	0.188375	0.064006	0.64944	0.24916	0.110924
8	Motivate	-0.17822	0.133053	0.451681	0.188375	0.914216	0.372199	0.852591	0.297269	0.021008
9	Caring	-0.18242	0.091036	0.239916	0.064006	0.372199	1.87388	0.362675	0.254412	-0.04454
10	Charisma	-0.07178	0.665266	3.197899	0.64944	0.852591	0.362675	6.705812	1.082143	0.157143
11	Passion	-0.15651	0.205882	0.578571	0.24916	0.297269	0.254412	1.082143	0.906933	0.04958
12	Friendly	-0.2563	0.134454	0.006723	0.110924	0.021008	-0.04454	0.157143	0.04958	0.346218

Диагонализированная матрица

	L	M	N	O	P	Q	R	S	T	U
3		Expect	Entertain	Comm	Expert	Motivate	Caring	Charisma	Passion	Friendly
4	Expect	1	-0.17658	0.079472	0.120531	-0.14721	-0.10525	-0.02189	-0.12979	-0.34401
5	Entertain	-0.17658	1	0.446921	0.048459	0.244122	0.116667	0.450687	0.37926	0.400869
6	Comm	0.079472	0.446921	1	0.152244	0.286424	0.106265	0.748758	0.368359	0.006927
7	Expert	0.120531	0.048459	0.152244	1	0.194286	0.046109	0.247318	0.258007	0.185906
8	Motivate	-0.14721	0.244122	0.286424	0.194286	1	0.284367	0.344343	0.326466	0.037342
9	Caring	-0.10525	0.116667	0.106265	0.046109	0.284367	1	0.102311	0.195155	-0.05529
10	Charisma	-0.02189	0.450687	0.748758	0.247318	0.344343	0.102311	1	0.438805	0.103132
11	Passion	-0.12979	0.37926	0.368359	0.258007	0.326466	0.195155	0.438805	1	0.088479
12	Friendly	-0.34401	0.400869	0.006927	0.185906	0.037342	-0.05529	0.103132	0.088479	1

Собственные векторы для каждого признака

	L	M	N	O	P	Q	R	S	T	U
18		2.880437	1.438654	1.16393	1.024453	0.705209	0.647623	0.562393	0.345248	0.232053
19	Expect	0.108673	0.639248	-0.2573	0.113776	0.417101	-0.09591	0.480829	-0.28424	-0.07242
20	Entertain	-0.41156	-0.25314	-0.1811	-0.26162	0.339636	6.8E-05	0.436453	0.592532	-0.09143
21	Comm	-0.44432	0.290679	-0.18728	-0.3022	0.030103	-0.13252	-0.3009	-0.06395	0.691799
22	Expert	-0.21564	0.135501	-0.18288	0.839641	0.028153	-0.00824	-0.22139	0.378794	0.057765
23	Motivate	-0.34066	0.036795	0.419631	0.156	-0.42352	-0.55994	0.424548	-0.08822	0.035156
24	Caring	-0.17657	0.025335	0.714513	0.062974	0.632544	0.029678	-0.21613	-0.07297	-0.0257
25	Charisma	-0.4815	0.199554	-0.1803	-0.16444	-0.08907	-0.09916	-0.36144	-0.16784	-0.70158
26	Passion	-0.40531	0.029434	0.107551	0.116215	-0.21505	0.782759	0.281837	-0.25913	0.06513
27	Friendly	-0.17762	-0.61785	-0.31904	0.23117	0.274429	-0.19028	0.041652	-0.55867	0.078217

Сортируем

	AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS	AT	AU	AV	AW
60	B										X		X'		Y
61	0.108673	0.639248	-0.2573	0.113776	0.417101	-0.09591	0.480829	-0.28424	-0.07242		2		-1.34917		0.782502
62	-0.41156	-0.25314	-0.1811	-0.26162	0.339636	6.8E-05	0.436453	0.592532	-0.09143		8		-0.29238		-1.96758
63	-0.44432	0.290679	-0.18728	-0.3022	0.030103	-0.13252	-0.3009	-0.06395	0.691799		1		-1.48548		-0.23406
64	-0.21564	0.135501	-0.18288	0.839641	0.028153	-0.00824	-0.22139	0.378794	0.057765		4		0.608124		1.123701
65	-0.34066	0.036795	0.419631	0.156	-0.42352	-0.55994	0.424548	-0.08822	0.035156		7		0.305044		-0.76562
66	-0.17657	0.025335	0.714513	0.062974	0.632544	0.029678	-0.21613	-0.07297	-0.0257		5		-0.7366		-0.66149
67	-0.4815	0.199554	-0.1803	-0.16444	-0.08907	-0.09916	-0.36144	-0.16784	-0.70158		4		-0.32502		-0.22281
68	-0.40531	0.029434	0.107551	0.116215	-0.21505	0.782759	0.281837	-0.25913	0.06513		4		-0.76129		0.149636
69	-0.17762	-0.61785	-0.31904	0.23117	0.274429	-0.19028	0.041652	-0.55867	0.078217		8		1.18966		-0.56694

Примеры применения PCA

- Визуализация многомерных данных:** снижение размерности до 2 или 3 для построения scatter plot.
- Предобработка перед классификацией:** используем PCA для уменьшения числа признаков, например, для логистической регрессии, SVM
- Предобработка перед кластеризацией:** используем PCA для уменьшения числа признаков, затем кластеризуем методом k -средних или иерархической кластеризацией

Реализация PCA на Python

- Реализация вручную (NumPy)**
 - Центрирование данных.
 - Матрица ковариации (`np.cov`).
 - Собственные значения и векторы (`np.linalg.eig`).

4. Сортировка и выбор K компонент.
5. Домножение исходных данных на матрицу компонент.

2. Использование Scikit-learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=K)
Z = pca.fit_transform(X)
```

| Обзор других методов снижения размерности

1. **Стохастическое вложение соседей с t-распределением (t-SNE, t-Distributed Stochastic Neighbor Embedding)**: нелинейный метод для визуализации высокоразмерных данных в 2D/3D. Хорош для визуализации кластеров, но не как общий метод извлечения признаков.
2. **UMAP (Uniform Manifold Approximation and Projection)**: алгоритм нелинейного снижения размерности данных, который эффективно визуализирует высокомерные данные в 2D или 3D, сохраняя локальную и глобальную структуру лучше, чем t-SNE.
3. **Автоэнкодеры**: нейросетевой аналог PCA (нелинейный). Encoder сжимает данные в латентное пространство, decoder восстанавливает их обратно.

| Лабораторная работа №2. Реализация метода главных компонент (PCA)

| Исходные данные

Используется набор данных **Breast Cancer Wisconsin** из `scikit-learn`:

- Документация: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
- Размерность: 30 признаков на объект (569 наблюдений)
- Задача: бинарная классификация (злокачественная/доброкачественная опухоль)

| 1. Загрузка и первичный анализ данных

1. Загрузить набор данных из Scikit-learn:

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
```

2. Загрузить в отдельные переменные:

- входные данные `X = data.data` (матрица признаков)
- выходные данные `y = data.target` (вектор меток)
- названия признаков `feature_names = data.feature_names`

3. Вывести:

- форму массивов `X.shape` , `y.shape`
- названия классов `data.target_names`
- первые 5 строк данных через pandas `DataFrame`
- базовую статистику по признакам (`describe()`)

4. Реализация PCA вручную (25 баллов)

| 2. Реализация PCA вручную

Реализовать функцию `my_pca(X, K)` , принимающую 2 аргумента:

- `X` — матрица данных (`numpy array` размера $m \times n$)
- `K` — искомое число главных компонент

Рекомендация: перед реализацией изучите [пример PCA на Excel](#).

Алгоритм функции должен включать следующие шаги:

| 2.1. Центрирование данных

Вычесть из каждого значения в столбце среднее значение этого столбца.

Использовать метод `X.mean(axis=0)` или `np.mean(X, axis=0)`

Формула: `X_centered = X - X.mean(axis=0)`

| 2.2. Построение ковариационной матрицы

Использовать метод NumPy `np.cov()`

Важно: указать параметр `rowvar=False` для корректной интерпретации строк/столбцов

Формула: `C = np.cov(X_centered, rowvar=False)`

Размерность результата: $n \times n$ (где n — число признаков)

| 2.3. Диагонализация матрицы

Найти собственные значения (eigenvalues) и собственные векторы (eigenvectors) методом `np.linalg.eig()`

Синтаксис: `eigvals, eigvecs = np.linalg.eig(C)`

`eigvals` — вектор собственных значений λ_i (размерность n)

`eigvecs` — матрица собственных векторов (размерность $n \times n$, каждый столбец — вектор)

2.4. Сортировка компонент

Отсортировать собственные значения и векторы по убыванию λ_i

Использовать метод `np.argsort()` с параметром `::-1` для обратного порядка

```
sorted_indices = np.argsort(eigvals)[-1:]
eigvals_sorted = eigvals[sorted_indices]
eigvecs_sorted = eigvecs[:, sorted_indices]
```

2.5. Выбор K главных компонент

Используя синтаксис слайсинга, оставить первые K столбцов из `eigvecs_sorted`

Формула: `W_K = eigvecs_sorted[:, :K]`

Размерность W_K : $n \times K$

2.6. Проекция данных

Домножить центрированные данные на матрицу компонент

Использовать метод `np.dot()` или оператор `@`

Формула: `Z = X_centered @ W_K` или `Z = np.dot(X_centered, W_K)`

Размерность Z : $m \times K$

2.7. Возврат результата (2 балла)

Функция должна вернуть:

1. `Z` — данные сниженной размерности ($m \times K$)
2. `eigvals_sorted` — отсортированные собственные значения (для анализа)

Итоговая структура функции:

```
def my_pca(X, K):
    X_centered = X - X.mean(axis=0)

    C = np.cov(X_centered, rowvar=False)

    eigvals, eigvecs = np.linalg.eig(C)

    sorted_indices = np.argsort(eigvals)[-1:]
    eigvals_sorted = eigvals[sorted_indices]
    eigvecs_sorted = eigvecs[:, sorted_indices]

    W_K = eigvecs_sorted[:, :K]
```

```
Z = X_centered @ W_K

return Z, eigvals_sorted
```

| 3. Применение вашей реализации PCA

Применить функцию `my_pca(X, k=2)` для получения проекции на первые 2 компоненты

Вывести:

1. форму результата `Z.shape`
2. первые 5 значений собственных чисел

Сохранить результат в переменную `Z2_manual`

| 4. PCA с использованием Scikit-learn

```
from sklearn.decomposition import PCA
pca_sk = PCA(n_components=2)
Z2_sklearn = pca_sk.fit_transform(X)
```

Вывести:

1. форму результата `Z2_sklearn.shape`
2. долю объясненной дисперсии: `pca_sk.explained_variance_ratio_`

| 5. Сравнение реализаций

Сравнить результаты вашей реализации (`Z2_manual`) и готовой (`Z2_sklearn`)

1. **Через 2 scatter plot** (визуальное сравнение)
2. **Численное сравнение** (через доли объясненной дисперсии)

| 6. Метод локтя и выбор оптимального числа компонент

Два графика построить:

1. **Scree plot (график осыпи)**
2. **График накопленной объясненной дисперсии:**

По графикам определить оптимальное число компонент

| 7. Визуализация в пространстве главных компонент

Для исходного набора данных построить scatter plot с раскраской по классам

Бонус: Провести эксперимент по сравнению качества логистической регрессии с PCA и без PCA (по F1-score и Accuracy)